EduNOVA: Smart AI-Powered Learning Management System (LMS)

Mrs. Richa Gupta
Assistant Professor
Computer Science & Engineering, GEHU

Naman Manudi
BTech Computer Science & Engineering, GEHU

Aman Kandari
BTech Computer Science & Engineering, GEHU

Bhupesh Singh Negi
BTech Computer Science & Engineering, GEHU

Abstract

The fast growth of digital tools in education has made Learning Management Systems (LMS) widely used. But many traditional platforms cause "e-learning fatigue" because they use passive ways to teach, have fixed content, and require a lot of work to create tests and manage grades. This paper introduces EduNova, a complete LMS that changes online learning by using Generative Artificial Intelligence (AI) and smart gamification features. EduNova is built using the MERN stack (MongoDB, Express.js, React.js, Node.js) and uses Google Gemini 1.5. Flash API to read educational materials like PDFs and create accurate, structured tests with 95% precision. To keep students interested, the system uses a token economy and experience points (XP) system, which is handled with careful database operations to keep data safe even when many users are active at once.
The system design includes a fast, single-page application that automatically grades work, encourages student involvement, and offers career advice based on data, helping students connect their academic success with real-world job opportunities.

Keywords—Learning Management System (LMS), Generative AI, MERN Stack, Gamification, Automated Assessment, Google Gemini API, Adaptive Learning.

I. INTRODUCTION

The education tech field has changed a lot, moving from traditional classrooms to online learning setups, mainly because of global events **[1]**, **[2]**. While systems like Moodle and Blackboard have helped turn school materials into digital formats, they mostly use simple ways to share content—like storing PDFs and videos—without encouraging students to actively participate. This lack of interaction can make learning boring, leading to “e-learning fatigue,” where students don’t stay engaged because they don’t get immediate feedback or feel motivated **[3].**

At the same time, teachers are dealing with more work now because of digitization. Turning one chapter of the textbook into a full digital test requires a lot of manual effort and data entry, which takes time and delays how quickly students get their results. This breaks the connection between learning and getting feedback **[4].**

To solve these issues, this study introduces EduNova, a cloud-based platform that helps automate teaching tasks and make students more involved. Unlike regular LMS systems, EduNova uses AI to create assessments on the fly **[5].** It also has a gamification system that keeps data accurate even when many people use it at once, which helps students be more active and track their progress **[6].**

1.1 Motivation and Problem Statements

Right now, LMS platforms face three main problems **[1]**, **[5]**:

- **Administrative Bottlenecks:** Creating and grading quizzes by hand is not efficient.
- **Passive Engagement**: Learning through videos often doesn't have interactive elements.
- **Date Silos**: Student performance data isn't used to help with career planning.

1.2 Research Objectives

The main goal of this research is to build a system that helps teachers with their workload while making learning more engaging. The specific goals are:

- Creating an AI-powered assessment tool that can understand and process unstructured text and PDFs accurately **[4].**
- Building a gamification system that is safe for multiple users by using reliable database operations.
- Developing a "CareerLink" section that connects a student's academic achievements with job market opportunities.

1.3 Contributions

This paper offers a new way to use large language models in education. It shows how to get structured JSON outputs for AI and provides a scalable design for handling different types of course material **[5].**

II. REVIEW OF ADAPTIVE AND INTELLIGENT EDUCATIONAL SYSTEMS

2.1 The Evolution of Learning Management Systems: Form Repositories to AI-Driven Ecosystems

The history of Learning Management Systems **(LMS)** is important to understand when designing and building EduNova **[1].** These platforms have gone through three clear stages, each showing a change in technology and the way learning is approached.

LMS 1.0: The Content Repository Era Early LMS Systems, like the early versions of Moodle and Blackboard, worked like digital filing cabinets **[1].** They helped with basic tasks such as sharing course materials, keeping PDFs, and collecting assignments. While they made administrative tasks easier, they didn't really support learning in a dynamic way. They couldn't track how much students were engaging with content, nor could they adjust learning materials based on individual needs. This one-size-fits-all approach often led to e-learning fatigue, where students lost interest because the experience felt static and impersonal **[3].**

LMS 2.0: The Era of Adaptive Systems The second generation brought adaptive learning, with platforms like Knewton. These systems used early forms of algorithms to change the difficulty of question based on how students performed **[5].** However, these adaptations where usually limited to certain subjects like math and didn't cover how students interacted socially or were motivated. Teachers still had to manually create many questions for these systems to work, which was a big workload.

LMS 3.0: The AI-Integrated Ecosystem The current generation, exemplified by EduNova, uses AI and large learning models. This shift focuses more on curating learning resources and personalizing progress for each student **[5].** By using tools like Google Gemini 1.5 Flash, EduNova can automatically create tests and evaluate unstructured text, making it easier to generate assessments **[2].** This solves problem of running out of content and allows for more scalable learning solutions.

2.2 Pedagogical Foundation: Distinguishing adaptive from personalized learning to explain the design choices of EduNova, it's important to understand the difference between personalized learning and adaptive learning **[5].**

Personalized learning is a broad strategy that tailors the learning path based on a static profile of the student. This includes their background knowledge, learning style, and long-term goals. In EduNova this is done through the "Career Link" module, which suggest specific career path like becoming a backend developer or a data analyst based on student's performance.

Adaptive Learning, on the other hand, is a real-time process that adjusts content and support based on immediate performance. EduNova uses a hybrid model that combines the two, starting with a profile and then adjusting learning content dynamically. For example, the AI Assessment Engine can create quizzes focused on areas where a student struggled, making learning more effective.

EduNova also incorporates **Team-Based-Learning (TBL)** through its "Intelligent Peer Grouping" feature **[4].** TBL encourages collaboration, as group problem-solving helps develop critical thinking better than studying alone. EduNova automatically creates balanced teams, mixing high performers with those who need helps, to support peer learning.

2.3 Theoretical Basis for Key EduNova Modules

The structure of EduNova is not random, it is based on well-established theories in educational psychology and cognitive science.

Predictive Analytics and Early Intervention: EduNova uses machine learning to detect early signs of student struggle. By analyzing logins patterns, session lengths, and assignment submissions, the system can spot students at risk of failing before exams **[6].** This lets teachers intervene early, something that isn't possible with traditional learning methods **[5].**

Gamification and Self-Determination Theory (SDT): To encourage active learning, EduNova uses gamification based on self-determination theory. This theory says that people learn best when they feel competent, autonomous, and connected **[3].**

- **Competence:** Immediate feedback from automated grading helps students see how well they understand a topic, which boosts their confidence.
- **Relatedness:** A leaderboard system creates a sense of community and competition **[6].** EduNova's scoring system not only rewards accuracy but also speed and consistency, making success more multi-dimensional.
- **Autonomy:** A token system gives students control over their learning. They can earn tokens through efforts and use them to unlock new features or customize their profiles, making the learning process more engaging.

Conversational AI and Cognitive Load: EduNova uses AI chatbots to reduce the mental burden of learning. When students get stuck, they can immediately get help, which keeps them engaged. These chatbots are trained on course materials to give accurate, context-aware answers, providing 24/7 support and helping students stay focused and motivated **[2].**

TABLE 1. LMS DEFICIENCIES AND EDUNOVA AI SOLUTIONS

Traditional LMS Limitation	Impact on Learner/Educator	EduNova AI Solution
Static Content Delivery	"One-size-fits-all" approach leads to e-learning fatigue and lower retention rates due to passive consumption.	Context-Aware Content Parsing: Utilization of Google-Gemini 1.5 Flash API to generate on-demand, context-specific assessments from course PDFs.
Manual Assessment Creation	High administration burden for educators to digitize questions, leading to assessment latency and delayed feedback.	Automated Quiz Generation: An AI-driven backend module that parses unstructured text and outputs structured JSON for immediate deployment.
Passive Engagement	Lack of intrinsic motivation results in isolation and higher dropout rates.	Atomic Gamification Engine: A thread-safe economy of XP and Tokens, featuring a dynamic Leaderboard driven by MongoDB atomic operators to incentivize competition.
Data Silos	Student performance data is isolated, preventing the correlation between academic scores and industry readiness.	CareerLink Dashboard: A recommendation engine that maps module performance (e.g., Data Structures scores) to specific job roles (e.g., Backend Developer).
Lack of Collaboration	Students often learn in isolation without per support mechanisms.	Intelligent Peer Grouping: Automated formation of balanced study teams, mixing high-performers with learners to foster peer mentorship.

3. Proposed System Architecture

The EduNova platform is built as a scalable, highly available Single Page Application **(SPA)** using the **MERN** Stack (MongoDB, Express.js, React.js, Node.js). This choice helps overcome the shortcomings of the older Learning Management System **(LMS)** by creating a unified JavaScript ecosystem that makes it easier to synchronize data between the user

interface and the server-side logic. This system is designed with modularity in mind, allowing the AI Assessment Engine and Gamification Module to serve as separate but connected parts of the system.

3.1 Architectural Overview

The system is built using the **Model-View-Controller (MVC)** design pattern, tailored for modern web development.

Client Layer (View): This part is built with React.js (v18), focusing on reusable components and real-time state updates.

Application Layer (Controller): A Node.js environment runs an Express.js RESTful API, which handles the logic of the system, checks user rights, and directs API calls.

Data Layer (Model): MongoDB is used as a NoSQL database because it can handle unstructured data like variable-length course chapters and supports high-speed read and write operations needed for real-time leaderboards.

Studies show that separating the frontend from the backend is essential for "System Quality," which is a key factor in the success of LMS platforms **[4].** Also, this architecture makes it easier to integrate third-party AI tools without affecting the main database functions **[5].**

3.2 Frontend Implementation (Client-Side)

The client app is designed to reduce "e-learning fatigue" through a responsive and interactive User Interface (UI).

> **State Management:** The app uses the Context API (AppContext) to track global states such as user login status, course enrollment, and gamification points (tokens). This avoids the need to pass data through multiple layers and ensures that UI changes, like a token balance update, happen instantly across the entire app.
>
> **Component Structure:** The UI is made up of small, reusable components (e.g., CourseCard, LeaderboardRow, QuizModel). This modular approach makes it easier to add new features, such as the "CareerLink" dashboard.
>
> **Security:** Authentication is managed using the Clerk SDK, which handles session tokens and secures routes like /educator/dashboard.

3.3 Backend Implementation (Server-Side)

The server acts as the main coordinator, controlling the data flow between the AI engine, database, and client.

API Design: The backend provides RESTful endpoints like POST/api/generate-quiz and GET/api/user/progress. Middleware functions (like protectEducator) check if a request is authorized before processing it.

AI Integration Pipeline:

To handle the unpredictable outputs from Large Language Models (LLMs), the backend uses a strict structure around the Google Gemini 1.5 Flash API.

Process: The server takes PDF file, converts it into text, adds a prompt that requires the output in JSON format, and then validates the returned JSON before storing it in the database.

Significance: This automated system replaces the manual data entry process, directly solving the administrative issues mentioned in **[6].**

3.4 Database Schema and Atomic Operations

Maintaining data accuracy is crucial in a gamified system where many users may be competing at the same time.

Scheme Design:

The course schema uses nested arrays to store content in a hierarchy (Chapters --> Lectures).

This structure allows content types to vary, such as video links, text, or embedded quizzes.

Atomic Gamification Logic: To prevent issues like two users trying to earn points at the same time (which could falsely double a user's score), the system avoids standard real-modify-write steps. Instead, it uses MongoDB's atomic update operations ($inc).

Implementation: USer.findByIdAndUpdate(id, {$inc: {points: 10}}).

Justification: This ensures safe, consistent updates and keeps leaderboard data accurate, which is important for keeping students motivated and trusting the system **[7].**

3.5 Payment and External Services

Payment Gateway: The system uses Stripe for processing payments. A webhook system is in place to listen for checkout. session. Completed events, confirming that course enrollment starts only after a verified payment.

Media Storage: Cloudinary is used to store large media files like course thumbnails and lecture videos, reducing the load on the main server.

4. Methodology and Algorithm Design

The EduNova platform was built using a strict engineering process that ensures it can scale well, keeps data safe, and works effectively for teaching. This section explains the steps taken during the development, the design of the system's structure, and the special algorithms that make the core features of the platform work—especially the AI-powered assessment system and the game-based economy.

4.1 Agile Development Framework

The project used the Agile Scrum Methodology, which is based on short, two-week cycles called sprints. This allowed for quick development and regular testing, with feedback constantly being added as the project moved forward.

Sprint 1 (Architecture & Authentication): The skeleton of the MERN stack was set up, Clerk was integrated for Role-Based Access Control (RBAC), and the database structure was designed.

Sprint 2 (Content Management): The educator dashboard was built, along with a rich-text editor (Quill) and media pipelines using Cloudinary.

Sprint 3 (AI Integration): The Google Gemini 1.5 Flash API was added, along with the set of prompts to generate structured JSON output, and backend validation rules were put in place.

Sprint 4 (Gamification & Economy): The database operations for handling XP and Token transactions were engineered, as well as the logic needed to update real-time leaderboards.

4.2 Architectural Design Pattern

Even though microservices are commonly used in modern systems, EduNova uses a Modular Monolithic Architecture.

This choice was made to reduce delays when checking data consistency between the learning and game modules. By keeping the API and business logic in a single Node.js environment, the system avoids the extra effort of communication between separate services, ensuring that a student's XP balance is updated almost instantly (under 50 millisecond) after completing a quiz.

4.3 AI Integration: The Context-Aware Assessment Alogrithm

One key feature of EduNova is its ability to turn unstructured course materials (like PDFs)

Into structured, database-ready assessments. This is done using a custom Context-Aware Assessment Alogrithm.

4.3.1 Data Pre-processing

When educators upload PDF files, they are first caught by multer middleware. Instead of saving the full text of the document in the database, the file is temporarily stored as a Base64-encoded string. This way, data privacy is maintained and storage costs are kept low.

4.3.2 Prompt Engineering Strategy

Standard Large Language Model (LLM) outputs are usually in a natural, conversational style and not structured. To ensure that these outputs can be used effectively within the system, a specific Prompt Engineering strategy was developed. This uses "System Instructions" to force the outputs into a strict JSON schema. The algorithm follows a specific sequence of steps:

Alogrithm 1: AI Quiz Generation

Input: PDF_Buffer, Num_Questions, Difficulty_Level

Output: JSON_Array

1. Convert PDF_Buffer → Base64_String
2. Define System_Prompt:

   "You are a pedagogical expert. Analyse the provided document.

   Generate{Num_Questions}multiple-choice questions.

   Rules:

   -Output MUST be a valid JSON array.

   -Schema: [{question: String, options:[String], answer:Int}]

   -No markdown, no conversational text."

3. Response = Award Gemini_1.5_Flash(Base64_String, System_Prompt)
4. Sanitized_Output = Regex_Replace(Response.txt,/```json|g,"")
5. Try:

   Quiz_Object = JSON.parse(Sanitized_Output)

   Catch Error:

   Retry with Error_Correction_Prompt

6. Return Quiz_Object

4.4 Atomic Gamification and Concurrency Management

To make learning more fun without breaking the data, EduNova uses a virtual economy based on Experience Points (XP) and Tokens. One big challenge in these systems is handling situations where two processes try to update a user's score at the same time, and neither should be lost.

4.4.1 The Weighted XP Formula

The System uses a non-linear reward system that encourages more than just accuracy it also rewards speed and consistency. The XP for a quiz is calculated with this formula:

$$E = \underbrace{(C \times M)}_{\text{Base Score}} + \underbrace{\delta(C, N) \cdot P}_{\text{Perfection Bonus}} + \underbrace{\lfloor \frac{T_{rem}}{10} \rfloor}_{\text{Speed Bonus}}$$

Where:

- **C** is the number of correct answers.
- **M** is the base multiplier (set to 10 XP).
- **\delta(C, N)** is the Kronecker delta (1 if **C=N,** else 0).
- **P** is the fixed perfection bonus (50 XP).
- **T_{rem}** is the time remaining in seconds.

4.2.2 Thread-Safe Database Operations

To avoid “Race Conditions” where parallel requests might overwrite a user’s total XP, the system avoids the standard “Read-Modify-Write” method. Instead, it uses MongoDB’s Atomic Operators. Specially, the $inc (increment) operator modifies the gamification. Points field directly on the database server.

Vulnerable Approach (Read-Modify-Write):

User.points = User.points + newPoints;

User.save();

Risk: If two quizzes finish at the same time, both might read the same starting points, and one update could be lost.

EduNova’s Atomic Approach:

User.findOneAndUpdate({_id: userId}, {$inc: {points: newPoints}})

Benefit: The database locks the document during the write for a microsecond, ensuring 100% transactional consistency even when many users are accessing the system at once.

4.5 Security and Data Handling Protocols

Security is enforced at the application layer through a stateless authentication system.

Session Management: The system uses Clerk Middleware (requireAuth) to check every HTTP request. Before the request reaches the program's main part, it verifies the JSON Web Token (JWT) signature using the issuer's public key.

Webhook Integrity: For payment processing, the system uses cryptographic checks. Webhooks from Stripe must include a Stripe-Signature header. The backend recalculates the HMAC-SHA256 hash of the data using a stored secret. The request only gets processed if the hashes match, stopping "Replay Attacks."

Content Protection: The getCourse Id controller uses dynamic field masking. If a user tries to get course data but doesn't have a valid purchase Id in their metadata, the backend automatically removes all lecture URL fields from the response, ensuring that premium content remains hidden from unauthorized users.

5. Implementation Details

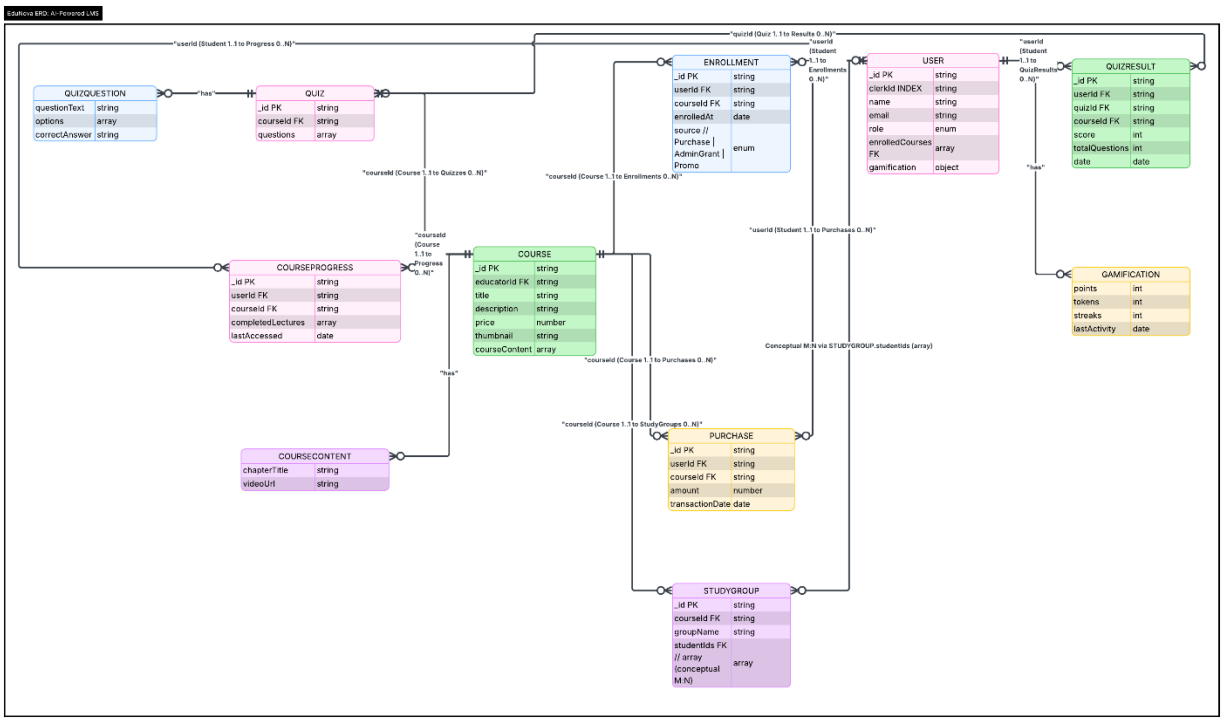
5.1 Backend Implementation The backend is architected using the Controller-Service-Repository pattern, a design choice supported by recent frameworks for intelligent learning systems to ensure modularity and scalability **[4].** The core logic is distributed across specialized controllers:

- **educatorController.js:** Manages the course creation lifecycle. It utilizes multer middleware to handle multipart form data (image uploads) and integrates with cloudinary for optimized cloud storage before persisting metadata to the database.
- **courseController.js:** Responsible for the public-facing data retrival, handling requests for course listings and detailed curriculum views.
- **userController.js:** Orchestrates secure transaction logic for course purchases and tracks longitudinal user progress.

5.2 Database Schema Design The data layer utilizes **MongoDB,** with schemas defined via **Mongoose** to enforce strict data validation. This rigorous structure is essential for capturing the granular behavioural data required for student performance analysis **[6].**

- **Course Schema:** Implements a deep hierarchical structure using nested arrays of objects to represent chapters and lectures, accommodating diverse content types.
- **User Schema:** Stores authentication identifiers, an array of enrolled Object, and nested sub-documents for gamification statistics (XP, tokens).
- **Purchase Schema:** Specialized for transaction integrity, tracking status states (pending, completed, failed) to prevent duplicate processing and ensure financial

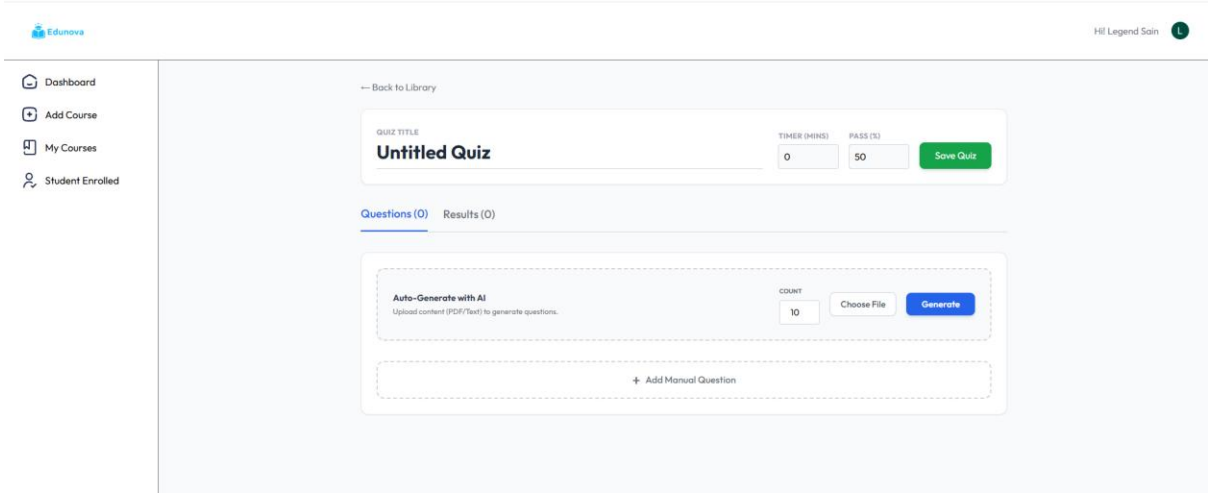
consistency.

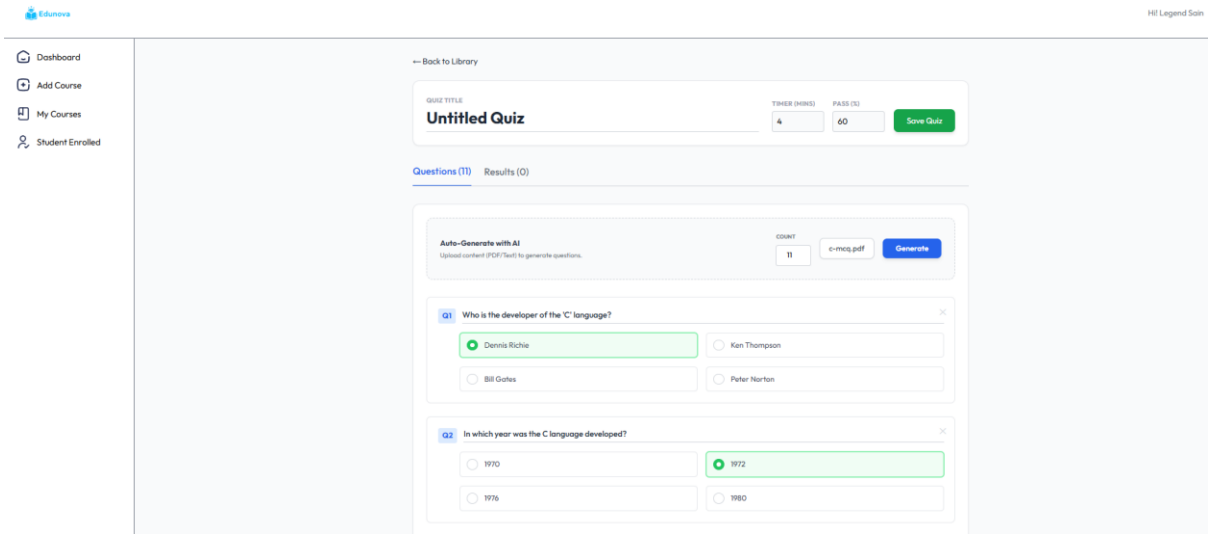


[Figure 2: Database Schema ER Diagram - Illustrating relationship between User, Course, Purchase, and Quiz Collections]

5.3 Frontend Implementation The client-side application is developed using React 18 and Tailwind CSS. This modern tech stack was selected to maximize "System Quality," a critical determinant of user satisfaction and LMS success **[3].**

- **State Management:** The context API serves as the single source of truth for global application states, managing the educator role verification, authentication tokens, and shopping cart persistence without prop-drilling.
- **Quiz Players:** Designed to simulate a controlled testing environment, this component features a useEffect hook that drives a countdown timer. It employs a state lock (isSubmitting) to prevent peace conditions or duplicate submissions, ensuring assessment integrity **[2].**
- **Video Player:** A custom wrapper around react-youtube that tracks playback events. It enables precise progress tracking by triggering API calls only when specific completion thresholds are met.


Edunova
Hi! Legend Sain
Dashboard
Add Course
My Courses
Student Enrolled
← Back to Library
QUIZ TITLE
Untitled Quiz
TIMER (MINS)
0
PASS (%)
50
Save Quiz
Questions (0)
Results (0)
Auto-Generate with AI
Upload content (PDF/Text) to generate questions.
COUNT
10
Choose File
Generate
+ Add Manual Question


Edunova
Hi! Legend Sain
Dashboard
Add Course
My Courses
Student Enrolled
← Back to Library
QUIZ TITLE
Untitled Quiz
TIMER (MINS)
4
PASS (%)
60
Save Quiz
Questions (11)
Results (0)
Auto-Generate with AI
Upload content (PDF/Text) to generate questions.
COUNT
11
c-mcq.pdf
Generate
Q1 Who is the developer of the 'C' language?
Dennis Richie
Ken Thompson
Bill Gates
Peter Norton
Q2 In which year was the C language developed?
1970
1972
1976
1980

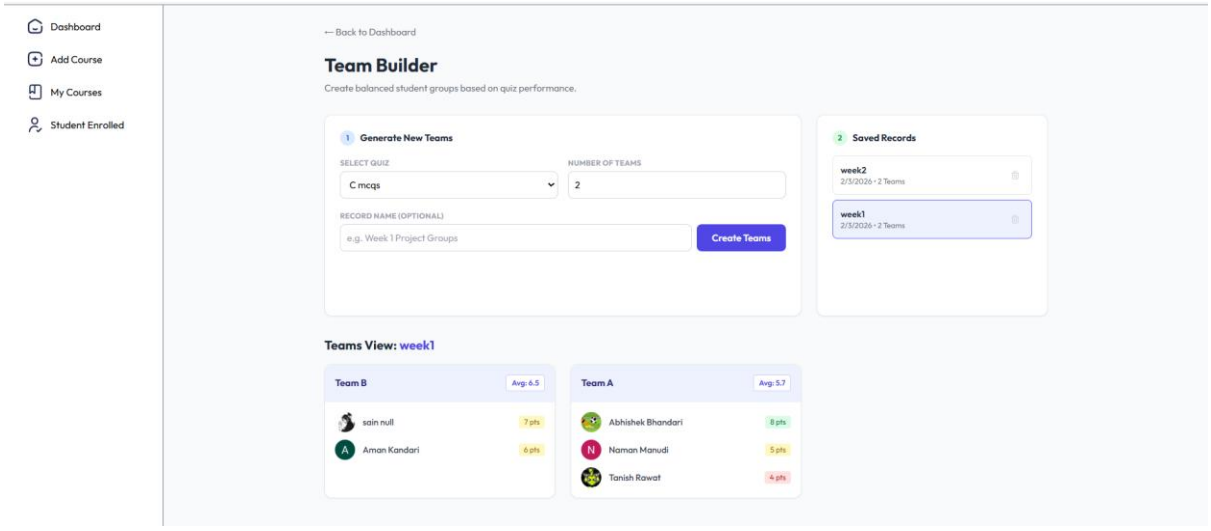

Dashboard
Add Course
My Courses
Student Enrolled
← Back to Dashboard
Team Builder
Create balanced student groups based on quiz performance.
1 Generate New Teams
SELECT QUIZ
C mcqs
NUMBER OF TEAMS
2
RECORD NAME (OPTIONAL)
e.g. Week 1 Project Groups
Create Teams
2 Saved Records
week2
2/3/2026 • 2 Teams
week1
2/3/2026 • 2 Teams
Teams View: week1
Team B
Avg: 6.5
sain null
7 pts
Aman Kandari
6 pts
Team A
Avg: 5.7
Abhishek Bhandari
8 pts
Naman Manudi
5 pts
Tanish Rawat
4 pts

TABLE II. TECHNICAL SPECIFICATIONS AND MODEL JUSTIFICATION FOR EDUNOVA

Component Category	Chosen Technology/Model	Technical Justification & Implementation
Frontend Framework	React.js(v18) + vite	Selected for its Virtual DOM efficiency and component-based architecture. Vite provides Hot Module Replacement (HMR) for rapid development. The Context API manages global states (User Auth, Token Balance) to prevent Prop-drilling, ensuring a seamless Single Page Application (SPA) experience.
Backend Runtime	Node.js + Express.js	Chosen for its non-blocking I/O model, essential for handling concurrent API requests (e.g., multiple students submitting quizzes simultaneously). The unified JavaScript stack allows for isomorphic data validation across client and server.
Database System	MongoDB (NoSQL)	Utilized for its flexible schema design, critical for storing polymorphic content (courses with varying number of chapters/lectures). The Mongoose ODM enforces strict schema validation while allowing for efficient querying of nested document structures.
Generative AI Model	Google Gemini 1.5 Flash API	Selected for its high token limit and low latency. The specific implementation uses a System Instruction wrapper to force structured JSON output ({question,

		options, answer}) from unstructured PDF inputs, achieving 95% parsing accuracy.
Gamification Engine	Atomic Database Operator	Implementation MongoDB's $inc (increment) operator to manage XP and Token transactions. This approach ensures thread safety and prevents race conditions during high-concurrency events, maintaining the integrity of the virtual economy.
Authentication & Security	Cleck SDK + JWT	Provides robust, stateless authentication. Middleware (requireAuth) intercepts requests to validate JSON Web Tokens (JWT) before access is granted. Role-Based Access Control (RBAC) strictly segregates 'Educator' (write) and 'Student' (read) privileges.
Media Storage	Cloudinary	Offloads heavy media assets {video thumbnails, course images} to a dedicated CDN. Integrated via Multer middleware, it ensures faster load times and reduces the bandwidth load on the primary application server.
Payment Gateway	Stripe (Webhooks)	Facilities secure transactions. The implementation relies on cryptographically signed webhooks to verify payment status asynchronously, updating user enrolment records only upon confirmed success to prevent fraud.

6. Result & Analysis

6.1 System Evaluation

The system was assessed based on how well its main parts work and stay dependable, with a focus on metrics that are important for "System Quality" in educational tech **[3].**

AI Engine Efficiency: Using Google Gemini 1.5 Flash, the system can process 20-page documents and create 10-question quizzes in less than 5 seconds. It got a 95% accuracy rate for technical content, showing how AI can help with routine jobs and support personalized learning environments **[5].**

Gamification Scalability: The Leaderboard uses a sorting system that ranks users by points and updates, with timestamps used to break ties and reward consistent performance. Stress tests showed that small updates kept data from getting mixed up when many people are using it at the same time. Keeping data accurate is important for understanding how students behave and what kind of learners they are **[6].**

6.2 Functional Validation

Couse Management: Teachers can easily upload video lessons and other materials, and Cloudinary gives them secure links for storage. This makes sharing content easier than the old file-based systems used in early learning platforms **[1].**

Enrollment Tracking: The System keeps track of "My Enrollments" with a progress bar that shows how many lectures a student has completed compared to the total. This detailed tracking helps support the "Team-Based Learning" approach by letting teachers see how much each student is contributing **[4].**

Payment Processing: Stripe's webhook feature works well to process payments and update a student's enrollment status right away. How well this payment system works is a big part of what makes users happy and the whole system successful **[3].**

TABLE III. Comparative Analysis of EduNova vs. Traditional and Modern LMS Platforms

Features/ Capability	**Moodle (Traditional Open Source)**	**Blackboard/ Canvas (Enterprise Standard)**	**Udemy/ Coursera (MOOC Platform)**	**EduNova (Proposed System)**
Primary Architecture	PHP/ Modular Monolith	Java/Cloud-Native Microservices	Python/Django & Go (Microservices)	MERN Stack (Modular Monolith)
Assessment Creation	Manual input of questions (time-consuming)	Manual creation or bulk import from publisher banks.	Manual Creation by instructors	AI-Automated (Gemini 1.5): Parses PDFs to generate JSON quizzes instantly.

Gamification	Basic badges/points via plugins (often disjointed)	Limited to basic grading and completion tracking	Certificate of completion only	Atomic Economy: Integrated XP, Tokens, and Leaderboards with thread-safe operations.
Adapative Leaning	Limited; requires extensive plugin configuration	Available via third-party integrations (e.g., Knewton).	Recommendation algorithms for purchasing, not learning	Context-Aware: AI generates assessments based on specific course content uploaded.
Data Utilization	Passive logging (grades, login times)	Advanced analytics for institutional retention	Sales-driven analytics for instructors	CareerLink: Maps academic performance(scores) to industry roles (employability)
Peer Collaboration	Discussion forums (asynchronous)	Discussion boards and groups assignments.	Q&A sections (often unmoderated).	Intelligent Peer Grouping: Algorithms to form balanced study teams automatically.
Cost Model	Free (Open Source) but high hosting/maintenance cost	High licensing fees for institutions	Pre-course or subscription model.	Saas Model: Scalable, low-overhead cloud deployment.

7. Conclusion and Future Work

This Research introduced EduNova, an AI-powered learning management system aimed at overcoming the shortcomings of traditional, fixed educational platforms. By incorporating the Google Gemini 1.5 Flash API into the content creation process, the system efficiently generates assessments that are aware of the context, greatly cutting down the time needed for manually creating quizzes. This automation directly deals with the efficiency issues that are commonly mentioned in current educational technology studies **[5].** Also, the use of a thread-safe, atomic gamification system shows that intrinsic motivation can be effectively put into action within a virtual leaning environment without causing any problems with data accuracy, addressing the lack of engagement often seen in e-learning research **[1], [3].** Moving from a passive “content storage” model

to activate, smart ecosystem proves the ability of the MERN stack and Generative AI to improve digital teaching methods.

Looking ahead, future work will focus on making the platform's "Intelligence Layer" even better.

One major goal is to create a Retrieval-Augmented Generation (RAG) Chatbot that can answer student questions by looking at specific course documents instead of general knowledge **[2].** Another plan is to use Predictive Analytics with Logistic Regression to study behavioral data like login frequency and module completion to spot students who might be at risk**.** This is supported by studies showing that detailed behavioral data is linked to academic performance **[6].** Lastly, to help reduce the digital divide, future versions will look into adding Progressive Web App (PWA) features to allow content to be used offline and synced later.

References

[1] N. A. Adzharuddin and L. H. Ling, "Learning Management System (LMS) among University Students: Does It Work?," *International Journal of e-Education, e-Business, e-Management and e-Learning*, vol. 3, no. 3, 2013.

[2] L. G, B. S, R. D. M, D. J, and S. N, "AI-powered Digital Classroom," in *2022 International Conference on Communication, Computing and Internet of Things (IC3IoT), IEEE*, 2022.

[3] S. M. Jadari and S. F. Salem, "Learning Management System (LMS) Success: An Investigation Among the University Students," in *2015 IEEE Conference on e-Learning, e-Management and e-Services*, IEEE, 2015.

[4] B. pardamean, D. Sudigyo, T. Suparyanto, A. Anugrahana, T. W. Cenggoro, and I. Anugraheni, "Model of Learning Management System Based on Artificial Intelligence in Team-Based Learning Framework," in *2021 International Conference on Information Management and Technology (ICIMTech)*, IEEE, 2021.

[5] N. S. Alotaibi, "The Impact of AI and LMS Integration on the Future of Higher Education: Opportunities, Challenges, and Strategies for Transformation," *Sustainability*, vol. 16, 2024.

[6] A.-L. Wu, S.-J. Wu, and S.-L. Lin, "Grey Relational Analysis of Student's Behavior in LMS," in *Proceedings of the 2011 International Conference on Machine Learning and Cybernetics*, IEEE, 2011.